

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторные работы №2 и №3 по курсу «Дискретный анализ»

Студент: Т. Д. Пупыкин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2

Задача: Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему собой регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64}-1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

- **+ word 34** — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.
- **- word** — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.
- **word** — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено. В случае, если слово в словаре не было обнаружено, вывести «NoSuchWord».
- **! Save /path/to/file** — сохранить словарь в бинарном компактном представлении на диск.
- **! Load /path/to/file** — загрузить словарь из файла.

Для всех операций, в случае возникновения системной ошибки, программа должна вывести строку, начинающуюся с **ERROR:** и описывающую на английском языке возникшую ошибку.

Вариант структуры данных: Красно-чёрное дерево.

Лабораторная работа №3

Задача: Для реализации словаря из предыдущей лабораторной работы, необходимо провести исследование скорости выполнения и потребления оперативной памяти. В случае выявления ошибок или явных недочётов, требуется их исправить.

Результатом лабораторной работы является отчёт, состоящий из:

Дневника выполнения работы, в котором отражено что и когда делалось, какие средства использовались и какие результаты были достигнуты на каждом шаге выполнения лабораторной работы. Выводов о найденных недочётах. Сравнение работы исправленной программы с предыдущей версией. Общих выводов о выполнении лабораторной работы, полученном опыте. Минимальный набор используемых средств должен содержать утилиту `gprof` и библиотеку `dmalloc`, однако их можно заменять на любые другие аналогичные или более развитые утилиты (например, `Valgrind` или `Shark`) или добавлять к ним новые (например, `gcov`).

1 Описание

Требуется реализовать собственный словарь на базе красно-чёрного дерева. В основе лежат специфические правила раскраски узлов и операции балансировки — левое и правое вращение, а также перекраска, которые гарантируют, что высота дерева не превысит $2 \log(n + 1)$.

Для этого реализуется класс `RBTree` и структура `Node`, включающая в структуру `Pair`, в которой хранится ключ, значение, указатели на потомков и структура данных `ParentWithColor`, которая хранит в себе ссылку на предка и цвет узла (RED/BLACK) с помощью техники *Pointer tagging*, позволяя использовать младшие биты для хранения информации.

А после надо провести исследование скорости выполнения и потребления оперативной памяти.

1 Color

Перечисление с жесткой типизацией (`enum class`), определяющее цвет узла. В качестве базового типа используется `bool`, что идеально подходит для последующей упаковки в младший бит указателя (0 для RED, 1 для BLACK).

Color	
RED	Константа, представляющая красный цвет узла
BLACK	Константа, представляющая черный цвет узла
<code>operator«</code>	Перегрузка оператора вывода для отладки состояния дерева

2 Pair

Структура данных, представляющая собой полезную нагрузку узла. Она связывает строковый ключ (телефон или слово) с его соответствующим числовым значением `uint64_t`.

Pair	
<code>std::string key</code>	Ключ словаря для лексикографического сравнения
<code>uint64_t value</code>	Значение, ассоциированное с данным ключом
<code>Pair(...)</code>	Конструкторы для инициализации данных
<code>operator«</code>	Форматированный вывод содержимого пары

3 ParentWithColor

Шаблонный класс, реализующий технику *pointer tagging*. Поскольку адреса объектов в куче выровнены, младшие биты указателя всегда нулевые. Мы используем самый младший бит для хранения цвета, что экономит память и улучшает локальность данных.

ParentWithColor<T>	
<code>uintptr_t data</code>	Числовое представление указателя, совмещенное с битом цвета
<code>struct ColorProxy</code>	Прокси-объект для удобного чтения и записи цвета через оператор <code>=</code>
<code>struct ParentProxy</code>	Прокси-объект для обращения к родителю и его разыменования
<code>color()</code>	Доступ к биту цвета узла
<code>parent()</code>	Доступ к указателю на родительский узел
<code>ptr() const</code>	Извлечение чистого адреса памяти без бита цвета

4 Node

Базовый узел дерева. Содержит данные, ссылки на детей и упакованный указатель на родителя. Использование `alignas` гарантирует корректное выравнивание для работы *pointer tagging*.

Node	
<code>Pair data</code>	Полезная нагрузка узла
<code>Node* left / Node* right</code>	Указатели на левое и правое поддеревья
<code>ParentWithColor<Node> pwc</code>	Упакованный родитель и цвет узла
<code>Node(const Pair&)</code>	Конструктор инициализации узла данными
<code>operator«</code>	Вывод узла (для отладки: данные + цвет)

5 RBTree

Класс `RBTree` реализует самобалансирующееся бинарное дерево поиска. Все основные операции выполняются за $O(\log n)$ благодаря поддержанию свойств красно-чёрного дерева.

Для упрощения логики используется специальный фиктивный узел `NIL`, представляющий листья дерева. Он всегда окрашен в чёрный цвет и заменяет `nullptr`.

Поля класса	
<code>Node* root</code>	Корень дерева
<code>Node* NIL</code>	Фиктивный листовой узел (sentinel)

Методы модификации и поиска	
<code>bool insert(const Pair& data)</code>	Вставка элемента с последующей балансировкой через <code>insertFixUp</code> .
<code>bool remove(const std::string& key)</code>	Удаление элемента по ключу с возможной балансировкой через <code>removeFixUp</code> .
<code>Node* search(const std::string& key)</code>	Поиск элемента (регистронезависимый).
<code>Node* search_impl(const std::string& key)</code>	Внутренняя реализация поиска (возвращает NIL при отсутствии).

Методы балансировки	
<code>void leftRotate(Node* x)</code>	Левый поворот вокруг узла <code>x</code> .
<code>void rightRotate(Node* y)</code>	Правый поворот вокруг узла <code>y</code> .
<code>void insertFixUp(Node* z)</code>	Восстановление свойств дерева после вставки.
<code>void removeFixUp(Node* x)</code>	Восстановление свойств дерева после удаления.

Служебные методы работы с деревом	
<code>Node* successor(Node* x)</code>	Поиск следующего по порядку узла.
<code>Node* minimum(Node* x)</code>	Поиск минимального элемента в поддереве.
<code>void clear(Node* node)</code>	Рекурсивное удаление поддерева.
<code>void setUpNIL()</code>	Инициализация фиктивного узла NIL.
<code>std::string to_lower(std::string)</code>	Приведение строки к нижнему регистру.

Сериализация	
<code>void save(const std::string& path)</code>	Сохранение дерева в бинарный файл.
<code>void load(const std::string& path)</code>	Загрузка дерева из файла.
<code>void save_impl(Node*, std::ofstream&)</code>	Рекурсивная сериализация (pre-order).
<code>Node* load_impl(std::istream&, Node*)</code>	Рекурсивная десериализация дерева.

2 Консоль

```

root@74b1f15b9ece:/workspaces/Discrete-Analysis-MAI# cmake -B build
root@74b1f15b9ece:/workspaces/Discrete-Analysis-MAI# cmake --build build
root@74b1f15b9ece:/workspaces/Discrete-Analysis-MAI# ./build/lab2/lab2

```

+ a 1
OK
+ A 2
Exist
A
OK: 1
-A
OK
a
NoSuchWord

3 Результаты исследования производительности

В данном разделе представлены результаты тестирования разработанного красно-черного дерева (RBTree) в сравнении со стандартным контейнером `std::map` и детальный анализ использования ресурсов.

1 Анализ потребления оперативной памяти (Valgrind)

Для проверки эффективности техники *Pointer Tagging* и контроля утечек памяти использовалась утилита Valgrind Memcheck. Сводные данные представлены ниже.

RBTree

```
==787525== HEAP SUMMARY:
==787525==      in use at exit: 122,880 bytes in 6 blocks
==787525==    total heap usage: 280,970 allocs,280,964 frees,20,425,944 bytes
allocated
...
786090 LEAK SUMMARY:
786090    definitely lost: 0 bytes in 0 blocks
786090    indirectly lost: 0 bytes in 0 blocks
786090    possibly lost: 0 bytes in 0 blocks
786090    still reachable: 122,880 bytes in 6 blocks
786090             suppressed: 0 bytes in 0 blocks
786090
786090 ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

std::map

```
==786090== HEAP SUMMARY:
==786090==      in use at exit: 122,880 bytes in 6 blocks
==786090==    total heap usage: 280,971 allocs,280,965 frees,18,178,304 bytes
allocated
...
==787525== LEAK SUMMARY:
==787525==    definitely lost: 0 bytes in 0 blocks
==787525==    indirectly lost: 0 bytes in 0 blocks
==787525==    possibly lost: 0 bytes in 0 blocks
==787525==    still reachable: 122,880 bytes in 6 blocks
==787525==             suppressed: 0 bytes in 0 blocks
==787525==
==787525== For lists of detected and suppressed errors, rerun with: -s
==787525== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Показатель	RBTree (Custom)	std::map (STL)	Разница
Total heap usage	18,178,304 bytes	20,425,944 bytes	−11.0%
Размер узла (Node)	64 bytes	72 bytes	−11.0%

Вывод по памяти: Кастомная реализация потребляет на 2.14 МБ меньше памяти. Отсутствие `definitely lost` подтверждает корректную работу деструкторов. Наличие `still reachable` (122.8 КБ) относится к внутренним буферам стандартной библиотеки ввода-вывода и не является утечкой программы.

2 Исследование скорости выполнения

Замеры времени проводились на операциях вставки, поиска и удаления ключей типа `std::string`. Для каждой реализации было проведено по 10 тестов по 1 000 000 и 2 000 000 операций и посчитано среднее значение. Тесты представляли из себя набор из случайных команд вставки, поиска и удаления в одинаковом количестве.

Кол-во операций	RBTree (us)	std::map (us)	Разница
1,000,000	2,213,091	2,253,288	+1.8% (быстрее)
2,000,000	4,482,729	4,726,862	+5.2% (быстрее)

3 Анализ производительности (Callgrind)

Для детального анализа производительности был использован инструмент Valgrind Callgrind, позволяющий измерить количество выполненных инструкций и выявить наиболее затратные участки программы.

summary: 9,958,524,374 Ir

Общее количество выполненных инструкций составляет $\approx 9.96 \cdot 10^9$.

Неочевидные источники затрат

Профилирование выявило, что значительная часть времени выполнения связана не с алгоритмами балансировки дерева, а с сопутствующими операциями:

- **Посимвольное сравнение строк:**
 - функции сравнения внутри `std::string` (например, `char_traits::compare`)
 - суммарно $\approx 10^8$ инструкций

- **Оператор сравнения строк:**

- `std::operator==`
- $\approx 1,000,000$ вызовов
- $\approx 1.08 \cdot 10^8$ инструкций

- **Внутренняя реализация поиска:**

- функция `search_impl`
- значительная доля времени уходит на последовательные переходы по узлам и сравнение ключей

- **Основной цикл программы:**

- функция `main`
- аккумулирует значительную часть инструкций за счёт генерации операций и работы с данными

Вывод по профилированию

Анализ показывает, что:

- Существенная доля затрат приходится на **посимвольные операции над строками**, а не на структуру дерева.
- Даже при асимптотике $O(\log n)$ фактическая стоимость операций определяется длиной ключей.
- Функция `search_impl` демонстрирует, что узким местом является не навигация по дереву, а сравнение ключей на каждом шаге.

Практический вывод: при использовании строковых ключей производительность определяется не только структурой данных, но и стоимостью операций сравнения, что важно учитывать при проектировании высоконагруженных систем.

4 Дневник выполнения работы

- **Этап 1:** Реализация базовой структуры дерева. Реализация логики поворотов, балансировки при вставке и удалении.
- **Этап 2:** Оптимизация `Node` с помощью *Pointer Tagging*.

- **Этап 3:** Рефакторинг, исправление недочётов и мелких ошибок. Проверка на утечки памяти с помощью `Valgrind`.
- **Этап 4:** Сравнение с `std::map` по скорости работы, по затрате памяти с помощью `Valgrind`. Анализ количества вызовов функций через `Callgrind` и оформление отчета.

5 Выявленные недочёты и исправления

В ходе разработки были устранены следующие проблемы:

1. **Оверхед памяти:** Изначально цвет хранился как `enum`, что увеличивало размер узла из-за *padding*. Внедрен **Pointer Tagging**, интегрировавший цвет в младший бит указателя на родителя.
2. **Копирование данных:** Выявлено избыточное копирование строк при вставке. Добавлена перегрузка методов с `std::string&&`.

6 Выводы

В ходе выполнения лабораторной работы по курсу «Дискретный анализ» была спроектирована и реализована эффективная структура данных — **сбалансированное красно-чёрное дерево**.

Основные результаты работы:

- Реализованы базовые алгоритмы поддержания баланса дерева: левое и правое **вращения**, а также каскадная **перекраска** узлов, что гарантирует логарифмическую сложность основных операций.
- Освоена техника *Pointer Tagging*: оптимизация потребления памяти путём хранения метаданных (цвета узла) в неиспользуемых младших битах указателя на родителя.
- Реализованы механизмы **бинарной сериализации** и десериализации, позволяющие атомарно сохранять и восстанавливать состояние дерева из внешних файлов.
- Проведён анализ качества кода с использованием инструмента `Valgrind` для контроля отсутствия утечек памяти и профилировщика `Callgrind` для оценки производительности критических участков алгоритма.

Список литературы

- [1] Кормен, Т. Х. Алгоритмы: построение и анализ : [пер. с англ.] / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн. — 2-е изд. — М. : Издат. дом «Вильямс», 2007. — 1296 с. — ISBN 5-8459-0857-4.
- [2] Красно-чёрное дерево [Электронный ресурс] // Википедия : свободная энциклопедия. — URL: https://ru.wikipedia.org/wiki/Красно-чёрное_дерево (дата обращения: 30.06.2025).
- [3] ГОСТ 7.1–2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М. : Изд-во стандартов, 2004. — 166 с.